
Hands-On Project: VM Health Checker



What You'll Build

A bash script that checks key system parameters—CPU usage, memory, and disk space—and reports health status.

Interactive Mode with 'explain'

The script accepts an 'explain' argument to provide a detailed, user-friendly summary of the VM's condition.

DevOps Impact

Automating health checks helps teams monitor system integrity and proactively detect issues before outages occur.

Step 1: GitHub Repository Setup



Create Your Repo

Start by creating a new GitHub repository named ``vm-health-check``. Initialize it with a README and a ``.gitignore`` for shell scripts.

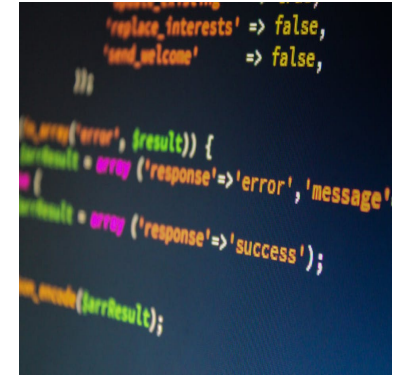
Folder Structure

Organize your files with a clear folder structure, e.g., ``.scripts/health_check.sh`` and ``.docs/usage.md``.

Version Control

Commit changes regularly with meaningful messages to track script improvements.

Step 2: Script Structure Overview



```
    'replace_interests' => false,  
    'send_welcome' => false,  
  })  
  try {  
    result = array ('response'=>'error', 'message'  
  ) (  
    result = array ('response'=>'success');  
  } catch (err) {  
    console.log(err);  
  }
```

■ Shell Script Entry Point

Create a `health\_check.sh` file with a `main()` function. Use modular functions for CPU, memory, and disk checks.

■ Argument Parsing

Support `--explain` using a `case` block or `getopts` to trigger a verbose summary mode.

■ Execution Flow

Check parameters > run health checks > generate output > optionally display explanations.

Step 3: CPU Health Check



CPU Usage Monitoring

Use ``top``, ``mpstat``, or ``/proc/stat`` to gather CPU usage. For example: ``top -bn1 | grep 'Cpu(s)'`.

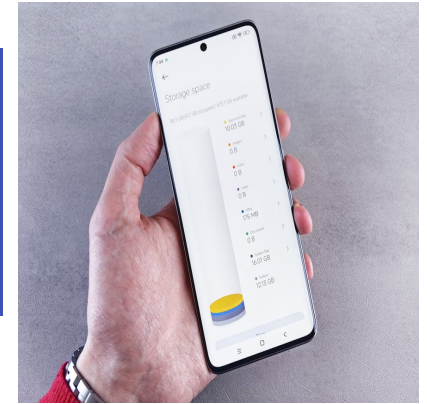
Set Thresholds

Compare current usage against a defined threshold (e.g., 85%) to determine health status.

Script Snippet

Example: ``if (( $(echo "$cpu > 85" | bc -l) )); then echo "High CPU usage"; fi``

Step 4: Memory Health Check



■ Capture Memory Usage

Use ``free -m`` to retrieve memory statistics and calculate usage:
``used/total` * 100.`

■ Set Safe Threshold

A usage over 90% might indicate memory pressure. Flag as 'Warning' or 'Critical' accordingly.

■ Script Snippet

Example: ``mem_usage=$(free | awk '/Mem/ { printf("%.2f", $3/$2 * 100.0) }')``

Step 5: Disk Space Check



Gather Disk Usage

Use ``df -h /`` or ``df --output=pcent / | tail -1`` to get the root filesystem's usage percentage.

Threshold Alerts

Mark disks as unhealthy if usage exceeds, say, 80%. Show warnings in red or bold terminal output.

Script Snippet

```
disk_usage=$(df / | awk 'END { print $5 }' | sed 's/%//')
```

Step 6: Combine All Health Metrics



Unified Health Logic

Run all three checks—CPU, memory, and disk—and aggregate their results into a single status report.

Visual Output

Use colors and symbols (✓ / ⚠ / ✗) to show system health per category.

Return Codes

Return specific exit codes (0: OK, 1: Warning, 2: Critical) to allow integration with cron or monitoring systems.

Step 7: Implement --explain Argument



Extended Output Mode

Add logic to detect `--explain` and display verbose descriptions for each metric's status.

Human-Friendly Language

Translate raw values into plain language. For example, 'CPU usage is above threshold. This may affect performance.'

Optional JSON or Markdown

Support output in structured formats for integration with monitoring tools or dashboards.

Step 8: Add Logging & Formatting



■ Log Important Events

Use ``logger`` or redirect output to a log file for tracking script execution and results.

■ Terminal Aesthetics

Use ANSI color codes for red (critical), yellow (warning), and green (healthy) statuses to improve readability.

■ Error Handling

Include fallback logic for missing tools or malformed inputs to make the script robust.

Step 9: Test Across VM Types



Run on Different Environments

Test the script on small, medium, and large VMs to ensure thresholds are realistic across contexts.

Validate --explain Output

Check if the extended output works as expected across varied loads and edge cases.

Automate Testing

Use cron jobs or CI pipelines to run and verify health checks periodically.

Step 10: Script Usage Examples



Basic Execution

Run ``./health_check.sh`` to print the current health status of the VM to the terminal.

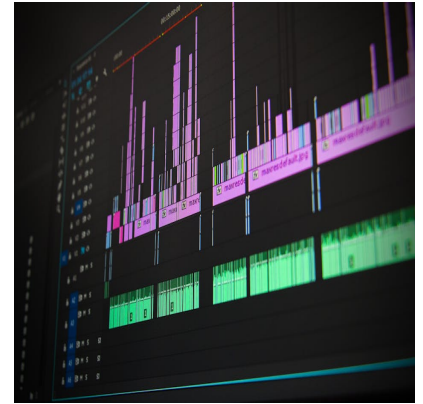
Verbose Mode

Run ``./health_check.sh --explain`` to get detailed output explaining each metric and its implications.

Automated Monitoring

Integrate into cron with ``*/15 * * * * /path/to/health_check.sh >> /var/log/vm_health.log``

Step 11: Automate Health Monitoring



■ Using Cron Jobs

Schedule the script to run periodically using ``crontab -e`` with your preferred timing interval.

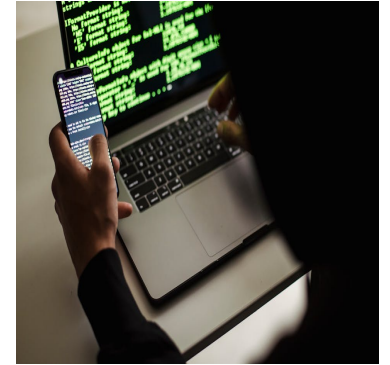
■ Email Alerts

Use ``mail`` or ``sendmail`` to notify ops teams of unhealthy statuses directly from the script.

■ Integrate with Dashboards

Log results in a format consumable by Prometheus/Grafana or custom monitoring UIs.

Step 12: Walkthrough Demo



Demo Objectives

Show how the script runs, outputs statuses, and responds to `--explain`.

Recording or Live Run

Use screen recording or live terminal share to walk through setup, usage, and output.

Validation

Confirm script behavior under high CPU, low memory, or full disk scenarios.